Machine Learning Classification of Lung Sound Recordings

Miguel Diaz-Alvarez, Jordan Mckoy, and AJ Carroll
Department of Electrical and Computer Engineering
University of North Carolina at Charlotte
Charlotte, NC, USA

***Abstract*—This report describes a machine learning methodology for classifying lung recordings. The study utilizes the HLS-CMDS dataset, which includes separate heart sound recordings, lung sound recordings, and mixed heart/lung recordings. The workflow consists of audio preprocessing, feature extraction, model training, and parameter tuning, which were done simultaneously using grid search. The best model from grid search was chosen based on the accuracy. Then a comparison of a multi-layer perceptron (MLP) and a convolutional neural network (CNN) was done using macro (average) metrics. MLP beat CNN on all metrics.**

***Index Terms*—machine learning, audio classification, heart sounds, lung sounds, feature extraction, neural networks**

I. Introduction

The objective of this project is to classify biomedical audio recordings into clinically relevant sound categories using machine learning techniques. Lung sounds are critical diagnostic signals, and automated classification can facilitate rapid screening, efficient data organization, and enhanced diagnostic decision support. This report addresses the challenges associated with accurate classification and summarizes the primary contributions of the study.

- preprocessing and organizing the HLS-CMDS audio dataset;
- extracting useful audio features for classification;
- training and comparing machine learning models;
- evaluating model performance using standard classification metrics.

II. Dataset

The dataset utilized for this study is derived from the HLS-CMDS dataset, which is organized into HS, LS, and Mix subsets. This analysis focused exclusively on the lung sound recordings from the LS and Mix subsets. In the mixed dataset, 45 lung sounds were shared across all mixed recordings. To eliminate duplicate sounds, a hash string was generated for each recording using the message-digest algorithm (MD5). Each hash string served as a unique fingerprint for its corresponding recording. These fingerprints were used to identify and remove duplicates. Following this process, 94 unique lung sound recordings remained across six classes.

Of the 195 labelled lung recordings present in the HLS-CMDS release, a large portion were exact duplicates of another file. Duplication was verified at the signal level as well as the byte level: for every flagged pair the Pearson correlation between waveforms was 1.000 and the maximum absolute sample difference was 0. Deduplication was performed *before* any train/test split, because a recording appearing in both partitions would cause the reported accuracy to reflect memorization rather than generalization.

TABLE I
Lung Sound Class Distribution

Label	Recordings
Pleural Rub	18
Rhonchi	17
Coarse Crackles	16
Normal	16
Wheezing	14
Fine Crackles	13
Total	**94**

All recordings were resampled to 8 kHz and converted to mono. As shown in Table **??**, the deduplicated dataset is close to balanced: 18 recordings in the largest class and 13 in the smallest, a ratio of approximately 1.38:1. This is mild by the standards of clinical datasets, and it bears on both metric selection and the resampling discussion in Section **??**. The dominant constraint in this study is not class imbalance but the small absolute size of the dataset.

III. Preprocessing and Feature Extraction

Before model training, the audio recordings were preprocessed to produce a consistent input representation. The following steps were applied to every recording:

- loading waveform files and labels as mono audio at 8 kHz using librosa;
- center-cropping or zero-padding each clip to a fixed duration of 10 seconds, giving 80,000 samples per recording;
- computing a mel-spectrogram with a 1024-point FFT, a hop length of 256 samples, and 64 mel bands;
- converting power to decibels, referenced to the per-clip maximum;
- standardizing each spectrogram to zero mean and unit variance.

The 8 kHz rate was selected because lung sounds are concentrated below 2 kHz, so the 4 kHz Nyquist limit retains all diagnostically relevant content while reducing input dimensionality. Fixing the clip length guarantees an identically shaped feature array for batching. The result is a 64×313 log-mel spectrogram, illustrated in Fig. **??**.

Two feature representations were derived from this common pipeline. For the MLP, each spectrogram was reduced to the mean and standard deviation of every mel band across time, producing a fixed 128-dimensional vector that summarizes the spectral envelope and its temporal variability. Reducing 20,032 spectrogram values to 128 features is substantial, and on a training set of 75 recordings this compression acts as a strong structural prior. For the CNN, the complete $1 \times 64 \times 313$ tensor was retained, preserving the local time–frequency structure that a convolutional model is designed to exploit: crackles and wheezes are defined by localized patterns rather than by average spectral level.

IV. Methodology

This section describes the machine learning models and training procedure used for the project.

A. Model Architecture

Two architectures were evaluated. The **multi-layer perceptron** was implemented with scikit-learn's `MLPClassifier` inside a `Pipeline` containing a `StandardScaler`, so that feature standardization is refit on each cross-validation training fold and never sees held-out data. The best MLP used one hidden layer of 64 units with tanh activation. Counting the output layer, that comes to 8,646 trainable parameters.

The **convolutional neural network** was implemented in PyTorch as three convolutional blocks with channel widths 16, 32, and 64. Each block applies a 3×3 convolution with padding of 1, batch normalization, and a ReLU activation; the first two are followed by 2×2 max pooling and the third by adaptive average pooling to a 1×1 extent. The result is flattened, passed through a dropout layer, and mapped to six outputs by a linear layer, giving 23,910 trainable parameters, nearly 3 times the MLP.

B. Training Procedure

Both models were trained on the 94 deduplicated recordings using stratified splits, so each class remained roughly proportional across the train and test sets. Without stratification, a small class such as the 13-example group could be poorly represented in the test set. The MLP used a 20% test split (75 training, 19 test), the CNN used a 25% split (70 training, 24 test), and the same random seed of 42 was used throughout for consistency.

Hyperparameters were tuned with `GridSearchCV` under stratified five-fold cross-validation on the training partition, scored by accuracy. The search spaces appear in Table **??**: 54 configurations (270 fits) for the MLP and 16 configurations (80 fits) for the CNN.

Three activation functions were compared for the MLP: the rectified linear unit, the hyperbolic tangent, and the logistic sigmoid. The search selected `tanh` with a single 64-unit hidden layer, $\alpha = 10^{-4}$, and an initial learning rate of 10^{-3}, reaching a best cross-validated accuracy of 0.880.

TABLE II
Hyperparameter Search Spaces

MLP (54 combinations)	
`hidden_layer_sizes`	(64,), (128,), (64, 32)
`activation`	relu, tanh, logistic
`alpha`	1e-4, 1e-3, 1e-2
`learning_rate_init`	1e-3, 1e-2
CNN (16 combinations)	
`lr`	1e-3, 1e-2
`optimizer__weight_decay`	1e-4, 1e-3
`module__dropout`	0.30, 0.50
`batch_size`	8, 16

The CNN was trained with cross-entropy loss, and the network produced raw output values that were converted into class probabilities by the loss function. Adam was used for optimization, with early stopping on a stratified 20% internal validation split and a patience of 10 epochs, capped at 100 epochs. To keep the comparison between models fair, the CNN was wrapped in a skorch `NeuralNetClassifier`, which exposes a scikit-learn compatible interface, so the identical `GridSearchCV` and `StratifiedKFold` procedure could be applied to both models over the same folds with the same scoring rule. The selected CNN used a learning rate of 10^{-3}, weight decay of 10^{-4}, dropout of 0.30, and a batch size of 8, reaching a best cross-validated accuracy of 0.671.

C. Evaluation Metrics

The trained models were evaluated using standard classification metrics:

- accuracy;
- precision;
- recall;
- F1-score;
- confusion matrix.

Since the deduplicated classes are nearly balanced, accuracy can be deemed a suitable metric. If said classes were strongly skewed, accuracy would not be a reliable metric. Accuracy alone is nonetheless insufficient. In single-label multi-class classification, micro-averaged precision, recall, and F1 are numerically identical to accuracy, so reporting them adds nothing. All figures in this report are *macro*-averaged, to give each class equal weight regardless of size. Under macro averaging a class the model never predicts contributes a zero rather than being absorbed by the remaining five, which proved decisive in interpreting the CNN results.

V. Results

Table **??** compares the two models. Both were tuned by the same grid-search procedure over the same cross-validation folds.

The MLP outperforms the CNN on every metric. A separate five-fold cross-validation of the tuned MLP over all 94 recordings gave per-fold accuracies of 0.842, 0.842, 0.895, 0.895, and 0.778, averaging 0.850 ± 0.043. The small spread

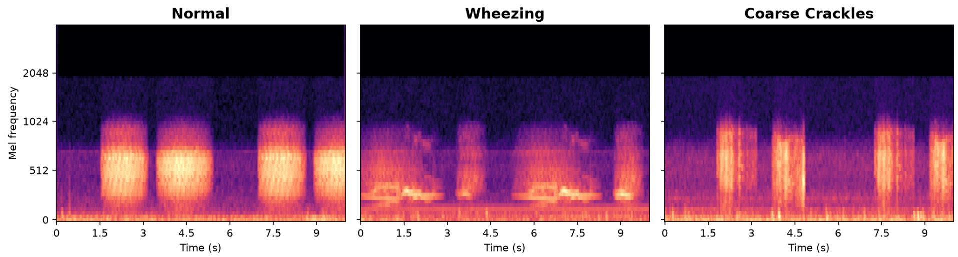

Fig. 1. Normalized log-mel spectrograms for three representative recordings. Wheezing produces sustained horizontal bands corresponding to narrow-band whistles, whereas coarse crackles appear as short vertical bursts distributed across frequency. Normal breath sounds show smooth, broadband respiratory cycles.

TABLE III
MODEL PERFORMANCE COMPARISON

Measure	MLP	CNN
Cross-validated accuracy (grid search)	**0.880**	0.671
Held-out test accuracy	**0.84**	0.75
Macro precision	**0.88**	0.66
Macro recall	**0.83**	0.72
Macro F1-score	**0.82**	0.68
Trainable parameters	8,646	23,910

in these results suggests that the 0.84 test accuracy is fairly representative, rather than the result of a lucky split.

Fig. **??** shows the same comparison graphically. The gap between the two models is wider under macro F1 (0.14) than under accuracy (0.09), because macro averaging makes the CNN pay in full for a class it fails to predict at all.

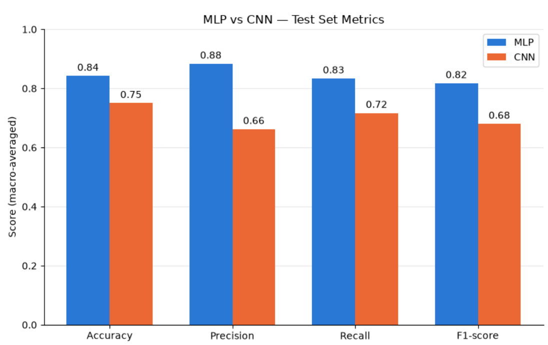

Fig. 2. Macro-averaged test-set metrics for the two models.

Per-class results are given in Table **??**. Wheezing was recovered perfectly by both models. Fine Crackles was the hardest class for both: the MLP recovered one of three test clips, and the CNN never assigned the label to any recording, producing precision, recall, and F1 of zero for that class.

The confusion matrices in Fig. **??** show where the errors occur. The MLP made three errors on 19 test recordings: two Fine Crackles clips were labelled Coarse Crackles and Rhonchi respectively, and one Normal clip was labelled Pleural Rub. The CNN made six errors on 24 recordings, and its errors are more concentrated: Pleural Rub was predicted seven times for five true instances, absorbing misclassified clips from three other classes.

One result is not captured by any aggregate metric. Inspecting the Normal column of both confusion matrices shows that every recording either model labelled Normal was in fact Normal; no abnormal recording was ever classified as healthy. The MLP's Normal recall of 0.67 reflects a Normal clip being flagged as Pleural Rub, which for a screening application is the benign direction of error.

VI. DISCUSSION

The MLP outperformed the CNN on every metric. A likely explanation is that the dataset is very small. The CNN had to learn its own feature detectors from only 70 training recordings, while the MLP was trained on a simpler representation based on the mean and standard deviation of each mel band. That made it easier for the MLP to perform well given the amount of data available. This trade-off between model capacity and training set size is a standard result in machine learning.

The class-level results support this interpretation. Wheezing was easy for both models, which is reasonable because wheezes appear as clear, sustained patterns in the spectrogram. Crackles were much harder. They are short, low-energy sounds, and the difference between coarse and fine crackles is subtle. For the MLP, the averaging used to create the 128 features likely removed some of the short-time details that define crackles. The CNN struggled even more, especially with Fine Crackles, and it often assigned uncertain cases to Pleural Rub.

There is little evidence that the MLP was overfit. Its cross-validated accuracy and its held-out test accuracy were very close, and the variation across folds was relatively small. The CNN showed a larger gap between its cross-validated and test accuracy, but with only 24 test recordings, that difference is still small enough that it should not be overinterpreted.

A. Class Balance and Resampling

A natural question is whether the small dataset could be improved with resampling. After deduplication, the largest class has 18 recordings and the smallest has 13, so the class imbalance is fairly mild. The bigger issue is not imbalance alone, but the fact that there are simply not many examples

TABLE IV
PER-CLASS TEST-SET PERFORMANCE

	MLP				CNN			
Class	**Precision**	**Recall**	**F1**	**Support**	**Precision**	**Recall**	**F1**	**Support**
Coarse Crackles	0.75	1.00	0.86	3	0.60	0.75	0.67	4
Fine Crackles	1.00	0.33	0.50	3	0.00	0.00	0.00	3
Normal	1.00	0.67	0.80	3	1.00	0.75	0.86	4
Pleural Rub	0.80	1.00	0.89	4	0.57	0.80	0.67	5
Rhonchi	0.75	1.00	0.86	3	0.80	1.00	0.89	4
Wheezing	1.00	1.00	1.00	3	1.00	1.00	1.00	4
Macro average	**0.88**	**0.83**	**0.82**	**19**	**0.66**	**0.72**	**0.68**	**24**

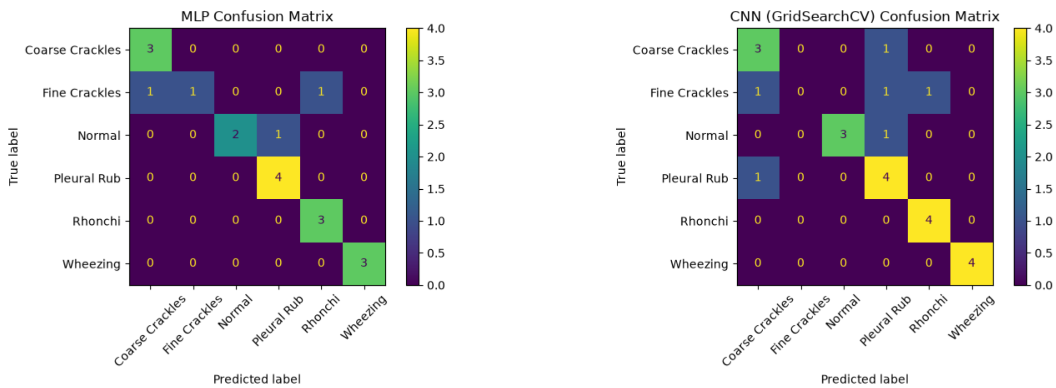

Fig. 3. Confusion matrices on the held-out test sets: tuned MLP (left, 19 recordings) and tuned CNN (right, 24 recordings).

overall. Resampling and augmentation could still help, especially for the harder classes such as Fine Crackles, but they are not a complete solution by themselves.

Oversampling in feature space: Random oversampling duplicates minority-class examples, while SMOTE creates new examples by interpolating between samples from the same class. This is straightforward for the MLP because its input is already a fixed 128-dimensional vector. For the CNN, though, the idea is less convincing because interpolating between two spectrograms does not necessarily produce a realistic lung sound.

Undersampling: Reducing every class to the size of the smallest would leave 78 recordings and discard 16 of the 94 available. On a dataset this small, that would probably remove useful information and hurt performance rather than help it.

Audio-domain augmentation: The most promising option is to augment the data in the audio or spectrogram domain instead of the feature domain. Label-preserving transformations such as time shifting, time stretching, pitch shifting, adding background noise, and SpecAugment-style time and frequency masking can create new examples that still look like realistic recordings. Because they are applied before feature extraction, they could help both the CNN and the MLP. This is especially relevant for Fine Crackles, which were the hardest class for both models.

A lighter alternative that does not create synthetic data is class weighting. Both frameworks support it directly—`class_weight` for scikit-learn estimators and the `weight` argument to `CrossEntropyLoss` for the CNN. This makes the model pay more attention to mistakes on minority classes without duplicating or fabricating samples.

Whichever approach is used, one important rule should be followed: resampling should happen inside the cross-validation loop and only on the training folds. Applying SMOTE before the train/test split would let synthetic points derived from test recordings leak into training, which would make the reported results too optimistic. That is the same kind of problem as the duplicate recordings discussed earlier, and it is easy to miss because the resampled data still looks well formed. The `imbalanced-learn` pipeline can combine resampling with an estimator in a way that is safe for cross-validation.

Any gain from these methods would also need to be judged carefully. With only three to five test recordings per class, a single correctly or incorrectly classified clip can change recall by 0.20 to 0.33. To show a real improvement, we would need repeated or nested cross-validation rather than a single held-out split.

B. Limitations

Several limitations affect how strong these conclusions are.

- **Dataset size.** Ninety-four recordings is very small. A test split of 19 to 24 clips means that one recording can change the accuracy by several points, so small differences between models are not very meaningful.
- **Single held-out split.** The test results come from one stratified split. The cross-validated results (0.850 ± 0.043 for the MLP) are a more reliable estimate of generalization.

- **Ordering sensitivity.** The file index is built by globbing directories, so the order of the files determines which copy of a duplicate pair is kept. An earlier run of the same code selected a two-layer $(64, 32)$ architecture instead of the $(64,)$ architecture reported here. Sorting the file list before deduplication would remove this variation.
- **No patient-level grouping.** Recordings are split at random, so two recordings from the same subject could end up on opposite sides of the split. Grouped cross-validation would be stricter and more clinically realistic.
- **Limited hyperparameter search.** The CNN grid covered 16 configurations and varied no architectural parameters beyond dropout. Filter counts, kernel sizes, and depth were fixed.

VII. Conclusion

This project builds a pipeline that classifies lung sound recordings into six categories. When creating this pipeline, cleaning and deduplicating the dataset proved to be very critical to ensure that the model was not memorizing the data. Of the 195 initial files, a large number were exact duplicates, leaving only 94 recordings. Had this step not been caught, accuracy would have looked deceptively high.

Our best MLP reached 0.880 cross-validated accuracy and 0.84 on the test set, with a macro F1 of 0.82. The CNN, tuned the same way, reached 0.671 and 0.75, with a macro F1 of 0.68. Of the three activations we tried, tanh worked best. Initial guesses led towards the CNN performing better due to spectrograms mirroring image classification, but the MLP overall outperformed the CNN, likely due to the 70 training recordings being too small to learn on.

In the future, to address this problem, we would use augmentation, which is oversampling done on the audio rather than on the features. Fine Crackles is the obvious place to start, since neither model handled it well and there are only 13 recordings of it. We should also have run that experiment instead of only reasoning about it.